PEANUT: Prompt-Enhanced Ablation with Optical Flow-Based Neural Unit for Spatio-Temporal Consistency & VSR++ Clarity from IMG2Video Field

M.H. Li (2330034027),
Beijing Normal University - Hong Kong Baptist University

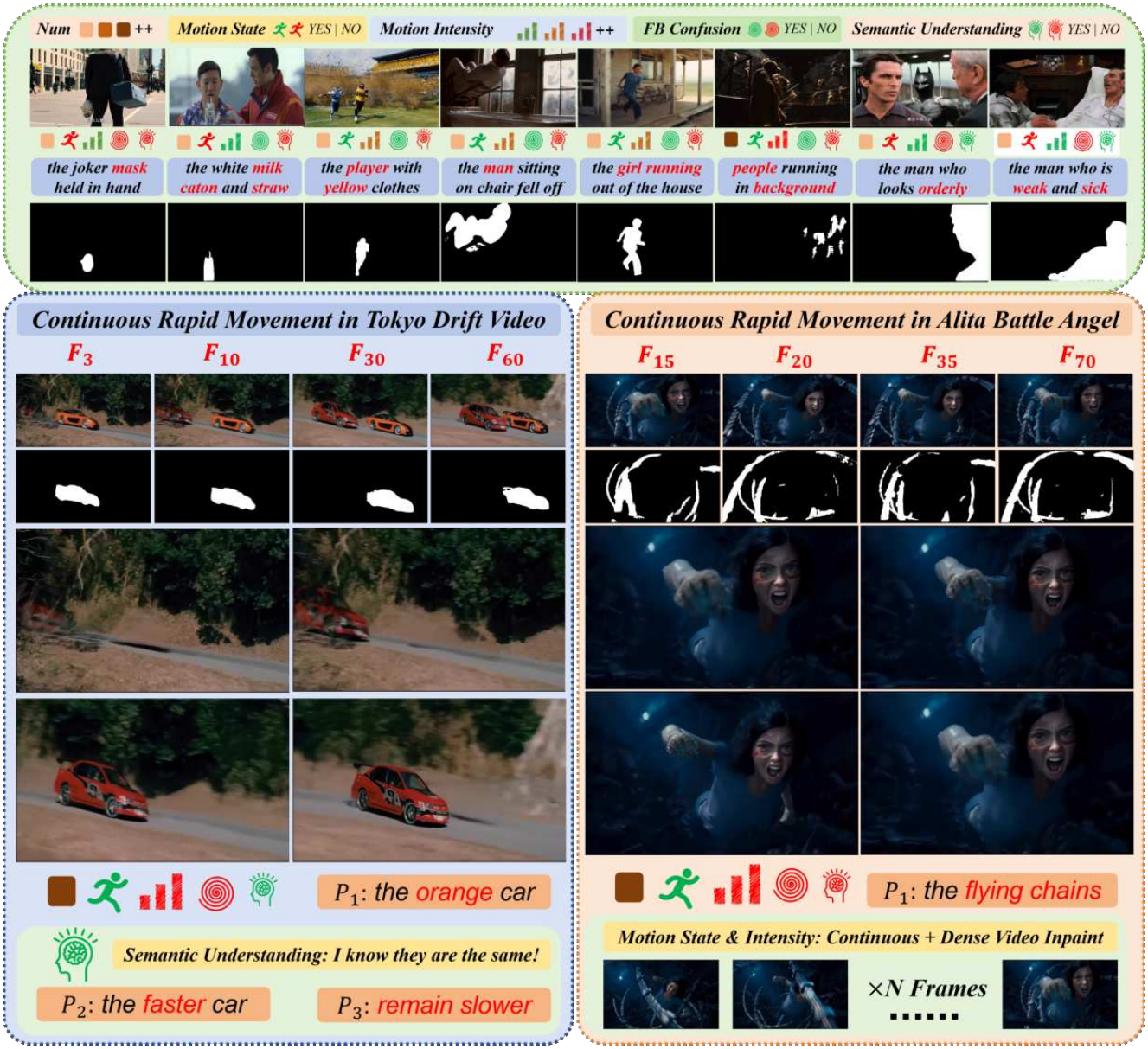


Figure 1. Final results visualization of PEANUT

Abstract

Video Object Removal (VOR) is a fundamental and challenging problem in computer vision, requiring precise object elimination and temporally consistent background reconstruction in dynamic scenes. To address the limita-

*tions of mask-dependent and frame-wise methods, we propose an efficient end-to-end **video-text grounded object removal** framework that directly maps textual descriptions to high-quality video outputs. The proposed approach evolves from 2D masked image inpainting to text-driven video editing by jointly modeling semantic grounding and temporal coherence. Specifically, a prompt-guided **P-MASk** module automatically generates per-frame object masks without manual annotations, while a **Conditional Memory Encoder (CME)** enhances robustness through conditional verification and cross-frame alignment. We further introduce **NOF-Eraser**, a flow-based removal module that combines optical-flow constraints with deep neural networks to ensure temporally smooth object removal, and integrate **BasicVSR++** to enhance the visual quality of the final output. The resulting unified pipeline achieves a favorable balance among performance, memory usage, and output quality, enabling practical deployment on consumer-level hardware. Codes and data are available at* `https://github.com/SuleynanAuir/PEANUT--Prompt-Enhanced-Ablation-with-Optical-Flow-Based-Neural-Unit-from-Image2Video-Field`.

1. Introduction

Video content editing is a fundamental yet challenging problem in computer vision. Among various tasks, Video Object Removal (VOR) aims to eliminate specified targets from dynamic video sequences while preserving spatial structure and temporal consistency, and plausibly reconstructing the regions occluded by the removed objects. This task plays an important role in applications such as film production, video postprocessing, and content creation. However, with the continuous increase in video resolution and duration, existing VOR methods still face significant challenges in terms of automation level, temporal stability, and computational efficiency.

Most existing video object removal methods formulate the task as frame-wise image inpainting, relying on manual masks or independent per-frame processing, which fails to capture long-term temporal dependencies and often results in temporal flickering and content drifting, especially in long or complex-motion videos. Although recent NeRF- and diffusion-based approaches improve visual quality, their prohibitive computational and memory costs limit practical scalability. Meanwhile, text-guided video editing has gained increasing attention with the advance of vision–language models; however, existing methods primarily target style transfer or object replacement and remain insufficient for stable, long-term object removal. The inherent semantic ambiguity of language, coupled with motion-induced temporal uncertainty and cross-frame localization errors, further complicates fully automated and temporally consistent video object removal.

To address these challenges, this paper investigates video object removal from the perspective of textconditioned, temporally consistent target elimination, and proposes an efficient, stable, and fully automated endtoend video object removal framework. Unlike conventional pipelines that follow a "maskfirst, then inpainting" paradigm, we move target localization and temporal consistency modeling to an earlier stage of the removal process. By leveraging text guidance to continuously constrain target regions across the entire video sequence, the proposed framework avoids the temporal instability caused by framewise independent processing.

In this paper, we first introduce a textguided Prompt-Guided Mask Generation (PMASk) module, which automatically generates framewise target masks according to natural language descriptions without requiring any manual annotations. To mitigate the disturbances introduced by frame rate variations and complex object motions, we further design a Conditional Memory Encoder (CME) that performs temporal constraint and correction on the generated masks through conditional verification and crossframe alignment mechanisms. Building upon this, we propose NOFEraser, a flowbased endtoend object removal module that combines explicit motion constraints with the representational capacity of deep neural networks, enabling spatially coherent and temporally consistent video object removal. Finally, we integrate BasicVSR++ at the output stage to further enhance video resolution and overall visual quality.

2. Related work

In this section, we review key technical areas relevant to video object removal, including video object segmentation, video inpainting, video super-resolution, and the construction of associated datasets. We focus on the details and limitations of existing approaches.

Referring Video Object Segmentation (RVOS) aims to segment specific objects in a video based on natural language instructions, called referring expressions. Early methods usually process videos in short clips or use a fully offline, batch mode. These methods work well on short videos, but they often lose global context in long videos[1]. As a result, object IDs can be lost when objects are occluded or reappear. Therefore, we aim to find a method that effectively addresses the attention drift caused by frame rate variations and achieves accurate frame-by-frame mask generation for objects in long-term videos.

The goal of video inpainting is to fill missing regions in a video, such as the masks of removed objects, with visually plausible content. Traditional methods are based on handcrafted patch-matching, which is computationally intensive and limited in semantic understanding. Early deep learning approaches use a multi-stage pipeline: they first estimate

optical flow, then propagate features across frames, and finally generate the missing content[2]. This separated workflow leads to error accumulation across stages and results in slow inference. In recent studies, it has been shown that by combining the accuracy of traditional optical flow with the generalization ability of deep neural networks, temporal and spatial consistency can be maintained while significantly improving inference efficiency[3].

Video super-resolution (VSR) is important to recover details lost during inpainting and to support high-resolution displays. Single-image super-resolution (SISR) ignores relationships between frames [4]. This often causes flickering in videos. Mainstream VSR models, such as EDVR, perform well. However, they have many parameters and are hard to run on consumer GPUs.

High-quality datasets are essential for evaluating video understanding and editing. Existing datasets have clear limitations. Popular datasets like DAVIS 2016 and YouTube-VOS provide dense annotations but have drawbacks: (1) they contain few frames and short videos, making it hard to test long-term stability; (2) they have low resolution, which is insufficient for high-definition editing; (3) their scenes and instructions are simple, lacking complex queries that test semantic understanding, such as multi object disambiguation [5].

3. Methodology

3.1. Datasets Preparation

To address the dataset limitations discussed in related work, we construct a new test set targeting real-world complex scenarios (In-the-Wild). The dataset follows two principles. First, semantic disambiguation: it contains many similar objects, such as multiple people in red, forcing the model to accurately interpret attributes, actions, or locations in the prompt instead of performing random removal. Second, high spatiotemporal complexity: it includes large motions, long-term occlusions, and complex background textures.By combining the diverse training data from YouTube-VOS [6] with our high-difficulty test set, we comprehensively evaluate model performance on multi-object disambiguation, temporal consistency, and fine-grained background reconstruction.

3.2. Model Overview

This work proposes a framework for end-to-end high-resolution video object removal guided by natural language instructions. We formalize the pipeline as a mapping function $\mathcal{F}$, which takes a video sequence $V = \{I_1, I_2, \ldots, I_N\}$ and a text description T as input, and produces the object-removed high-resolution video $\hat{V}_{HR}$ as output.

The system consists of three coupled modules:

Semantic Mask Generation (P-MASk) This module will generate temporally consistent binary masks $M = \{m_1, m_2, \ldots, m_N\}$ guided by the text.

Flow-Guided Object Erasing (NOF-Eraser) NOF-Eraser fills missing regions based on the masks and optical flow, producing a low-resolution inpainted video $\hat{V}_{LR}$.

Super-Resolution Enhancement (UR-Net) UR-Net maps $\hat{V}_{LR}$ to the high-resolution space $\hat{V}_{HR}$ and restores high-frequency details.

The full pipeline can be expressed as:

$$\hat{V}_{HR} = \text{UR-Net}(\text{NOF-Eraser}(V, \text{P-MASk}(V, T))). \quad (1)$$

3.3. P-MASk with CME

To address the limitations of manual annotations in traditional methods, we design the Prompt-Guided Mask Generation (P-MASk) with Cross-Modal Enhancement (CME) module. This module tackles the Referring Video Object Segmentation (RVOS) task, focusing on aligning textual and visual features across the spatio-temporal dimension.

3.3.1. Feature Extraction and Cross-Modal Fusion

Given the frame image I_t and text prompt T, visual features F_v^t are extracted using a visual encoder such as ResNet or Swin. Text embeddings F_l are obtained through a language encoder such as BERT or RoBERTa. The two feature sets are then combined using a cross-modal attention mechanism to produce the multimodal feature map F_{vl}^t.

3.3.2. Conditional Memory Encoder (CME)

To address the attention drift problem in long-term object tracking, where the model may mistakenly focus on similar non-target objects, we propose the CME module. CME maintains a dynamic memory $\mathcal{H}$ that stores feature prototypes of the target from previous frames. For the current frame t, we compute the matching between candidate object features f_{obj}^t and text, while applying a confusion check. The consistency score with the memory is computed as

$$S_{consistency} = \frac{f_{obj}^t \cdot \text{Read}(\mathcal{H})}{\|f_{obj}^t\| \|\text{Read}(\mathcal{H})\|}. \quad (2)$$

To filter false positives, we introduce a dynamic threshold τ. The current frame mask m_t is considered valid and used to update the memory only if $S_{consistency} > \tau$. Otherwise, a suppression mechanism corrects the mask using information from neighboring frames via optical flow from $t-1$. This ensures mask generation remains focused on the target defined by the text T.

3.4. NOF-Eraser

After obtaining the mask M, we treat the task as video inpainting. Unlike traditional methods that separate optical

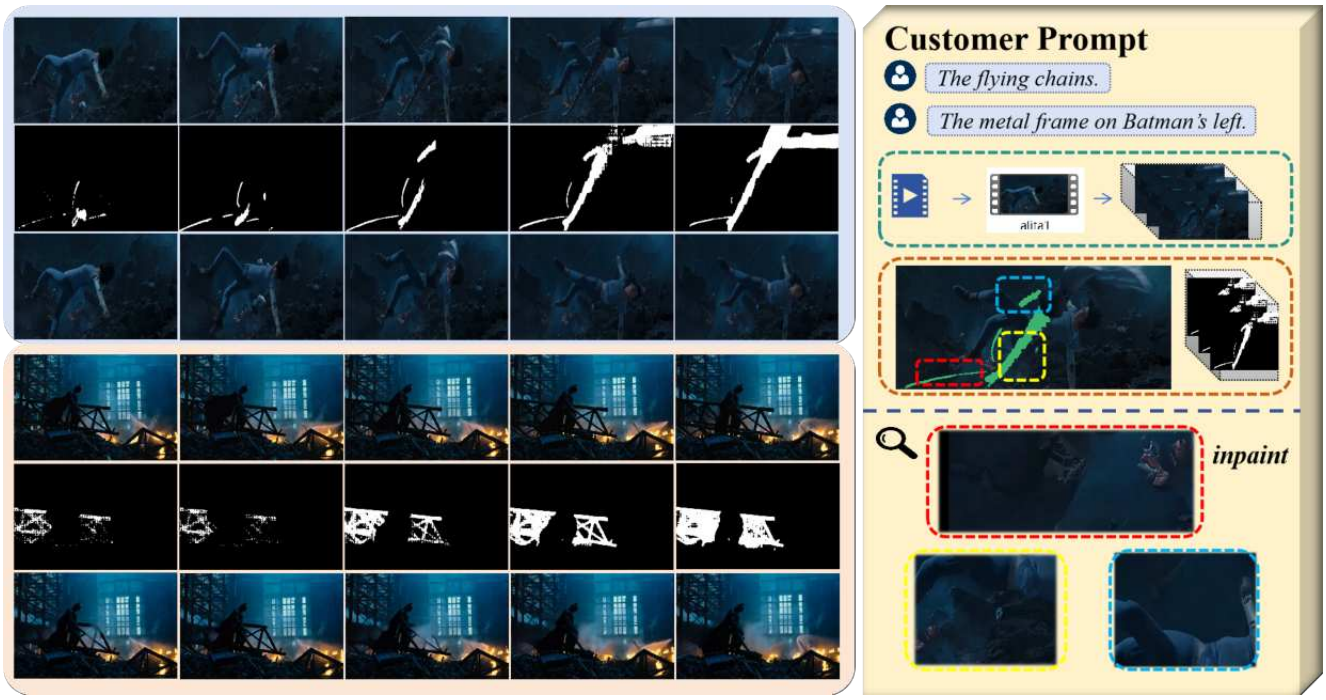


Figure 2. Examples of datasets (In-the-Wild)

flow estimation and pixel propagation [7], we design End-to-End Flow-Based Removal Module (NOF-Eraser), an improved version of the E2FGVI (End-to-End Flow-Guided Video Inpainting) architecture.

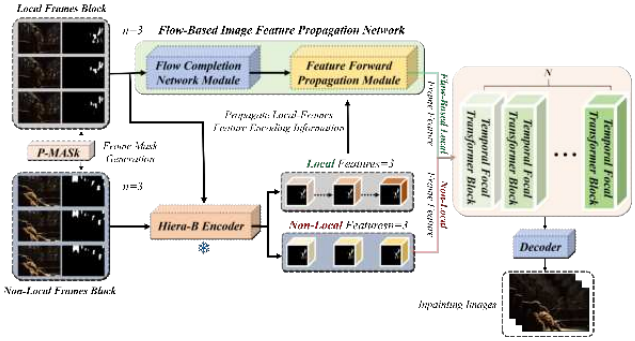


Figure 5. The structure of NOF-Eraser

3.4.1. Flow Completion Network

Since the optical flow in masked regions is unknown, we first hallucinate the missing motion. The inputs are the masked images $I_t \odot (1-m_t)$ and $I_{t+1} \odot (1-m_{t+1})$. The network predicts the forward flow $F_{t \to t+1}$ and backward flow $F_{t+1 \to t}$. We enforce flow smoothness by minimizing the loss $\mathcal{L}_{flow}$, ensuring that the motion is physically plausible and free of artifacts.

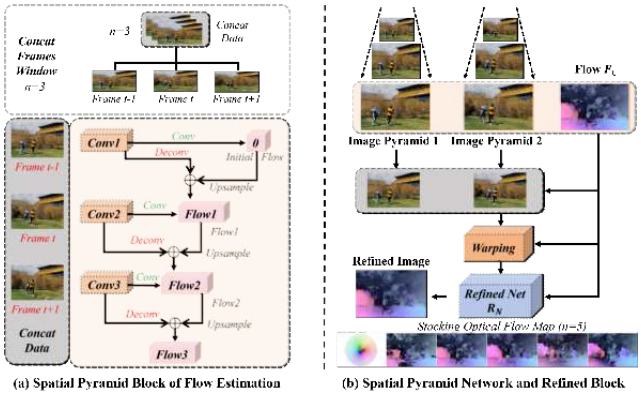


Figure 6. Schematic Diagram of Spatial Pyramid Architecture and Refinement Module for Optical Flow Estimation.

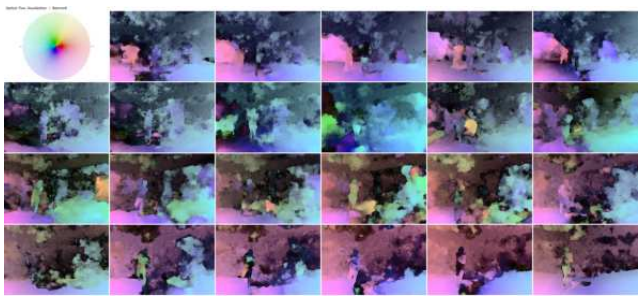


Figure 7. Example of Stacking Optical Flow Map

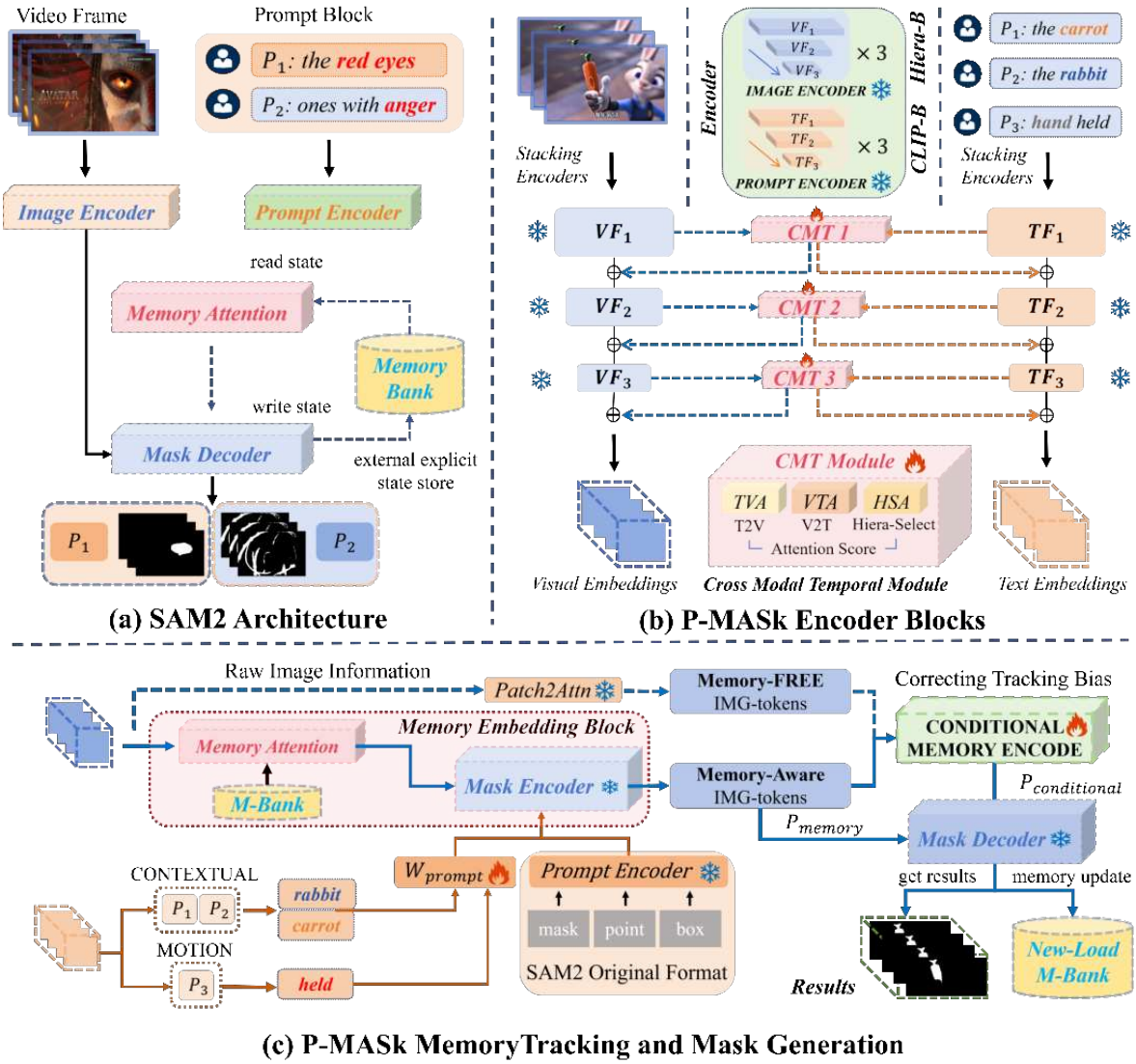


Figure 3. The structure of P-MASk

3.4.2. Feature Propagation and Content Hallucination

Using the predicted optical flow, we perform deformable alignment in the feature space.For frame,the features are aggregated temporally via bidirectional propagation:

$$Features_t^{prop} = \text{Warp}(Features_{t-1}, F_{t\to t-1}) + \text{Warp}(Features_{t+1}, F_{t\to t+1}). \quad (3)$$

The Content Hallucination Module then applies 3D convolutions on the aggregated features to generate the final inpainted frame $\hat{I}_t$. This design preserves temporal consistency through optical flow and spatial details through CNN, enabling end-to-end optimization within a single computation graph.

3.5. Video Super-Resolution Reconstruction (UR-Net)

To balance performance and quality, we use an improved BasicVSR++ module. Flow-guided deformable convolutions align features and leverage spatiotemporal information to restore details [8]. The lightweight design keeps computational cost low, allowing efficient high-resolution video output on devices such as an RTX 4070 Laptop.

3.5.1. Second-order Grid Propagation

To address minor jitters in inpainted videos, UR-Net employs a second-order grid propagation mechanism. Unlike simple unidirectional connections, this mechanism allows information to flow bidirectionally in the spatiotemporal grid. As a result, the reconstruction of frame t depends not

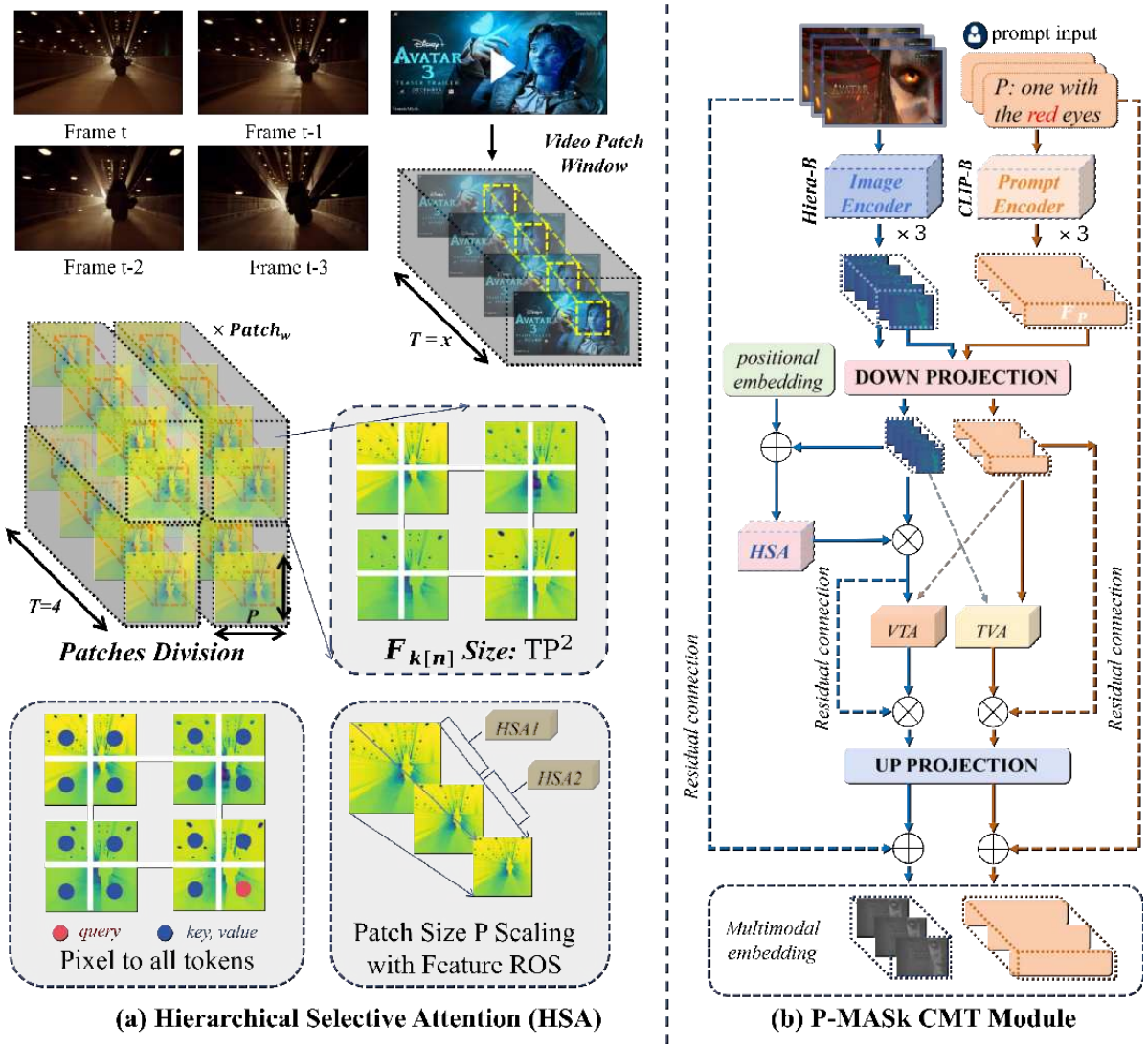


Figure 4. Hierarchical Selective Attention (HSA) in video understanding and the P-MASk Cross-Modal Transformer module structure diagram.

only on $t \pm 1$ but also on a longer temporal context $t \pm k$.

3.5.2. Flow-Guided Deformable Alignment

We reuse the intermediate optical flow features from NOF-Eraser in the UR-Net alignment module. This reduces redundant computation, lowering inference time, and ensures consistency in motion estimation between inpainting and super-resolution. The high-resolution frame is computed as

$$\hat{I}^t_{HR} = \text{Upsample}(\text{Conv}_{res}(\text{AlignedFeatures}_t)). \quad (4)$$

UR-Net outputs a $4\times$ upsampled video, enabling efficient processing of $1080p$ or $4K$ videos with limited memory (32G).

3.6. Loss Functions

To jointly train the modules, we use a combined loss:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{recon} + \lambda_2 \mathcal{L}_{adv} + \lambda_3 \mathcal{L}_{perceptual} + \lambda_4 \mathcal{L}_{flow} \quad (5)$$

$\mathcal{L}_{recon}$ (L1 Loss) ensures pixel-level reconstruction accuracy. $\mathcal{L}_{adv}$ (Adversarial Loss) employs a T-PatchGAN discriminator to improve the realism of generated textures. $\mathcal{L}_{perceptual}$ measures the distance between high-level semantic features extracted by VGG-19 to maintain perceptual quality. $\mathcal{L}_{flow}$ enforces smoothness in the optical flow field, reducing temporal jitter.

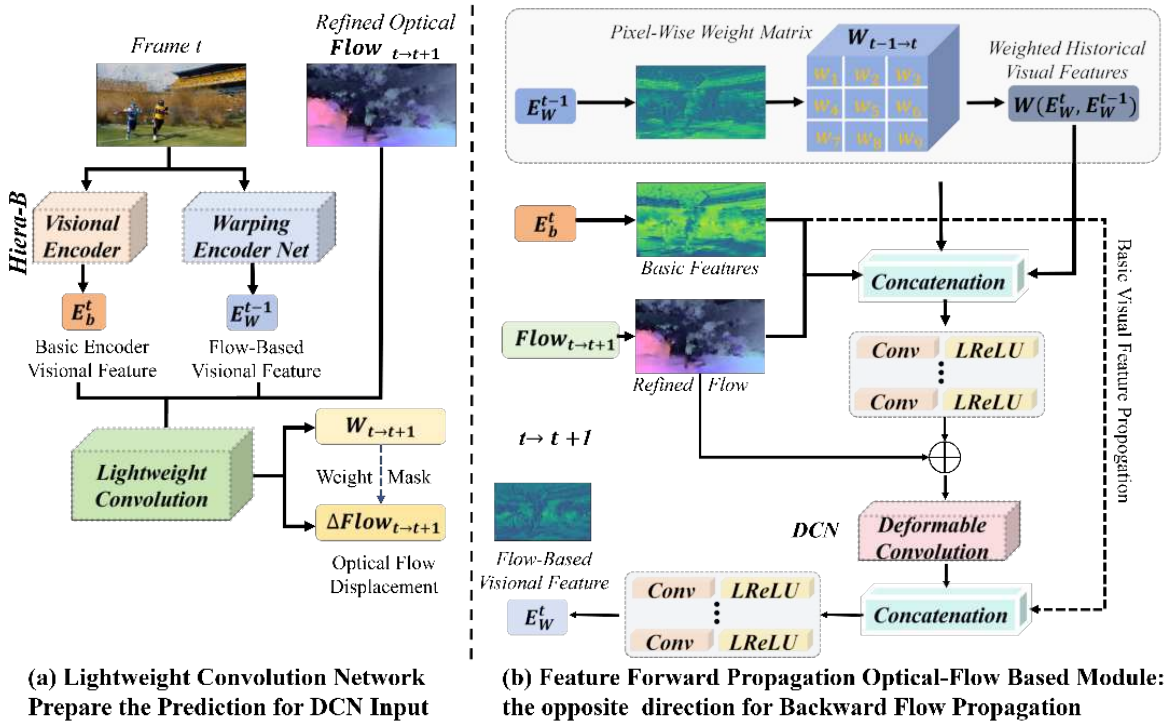


Figure 8. Structural Diagram of Optical Flow-Guided Feature Propagation Module. (a) depicts a lightweight deformable convolution module for feature alignment, and (b) shows the corresponding bidirectional optical flow propagation framework.

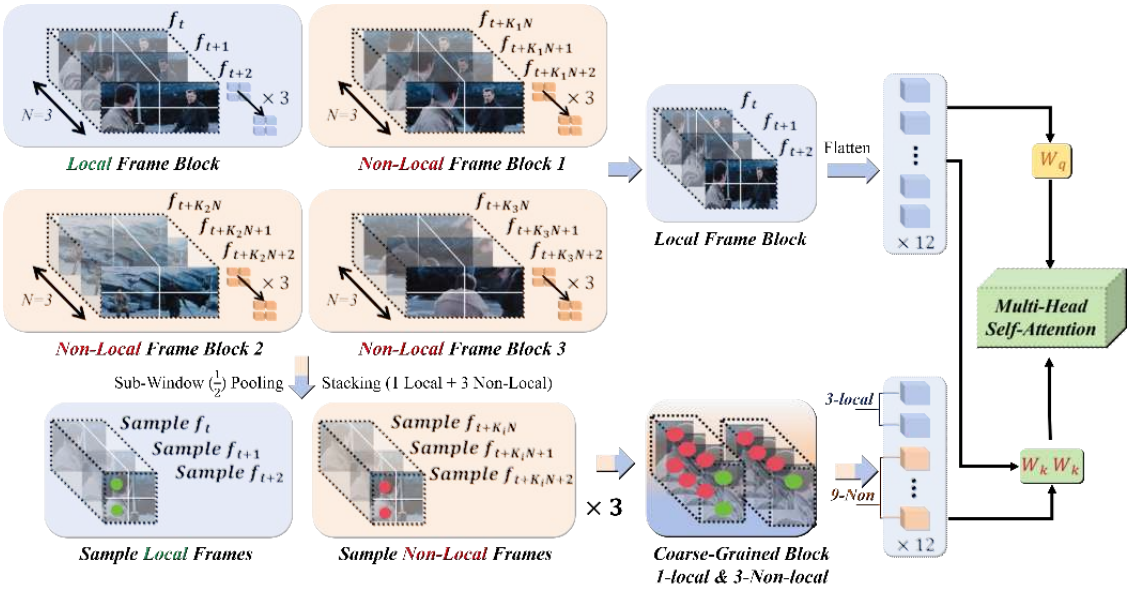


Figure 9. Architecture Diagram of Local and Non-Local Frame Feature Processing

4. Experiment

4.1. Big-LaMa Failure for Video Inpaint

We previously achieved strong results using Big-LaMa for single-image 2D inpainting. One might wonder if applying Big-LaMa directly to sampled video frames could achieve similar results, since dense sampling should in principle remove the target object perfectly. However, this approach does not perform as expected.

The first issue is artifacts. In dynamic scenes or with

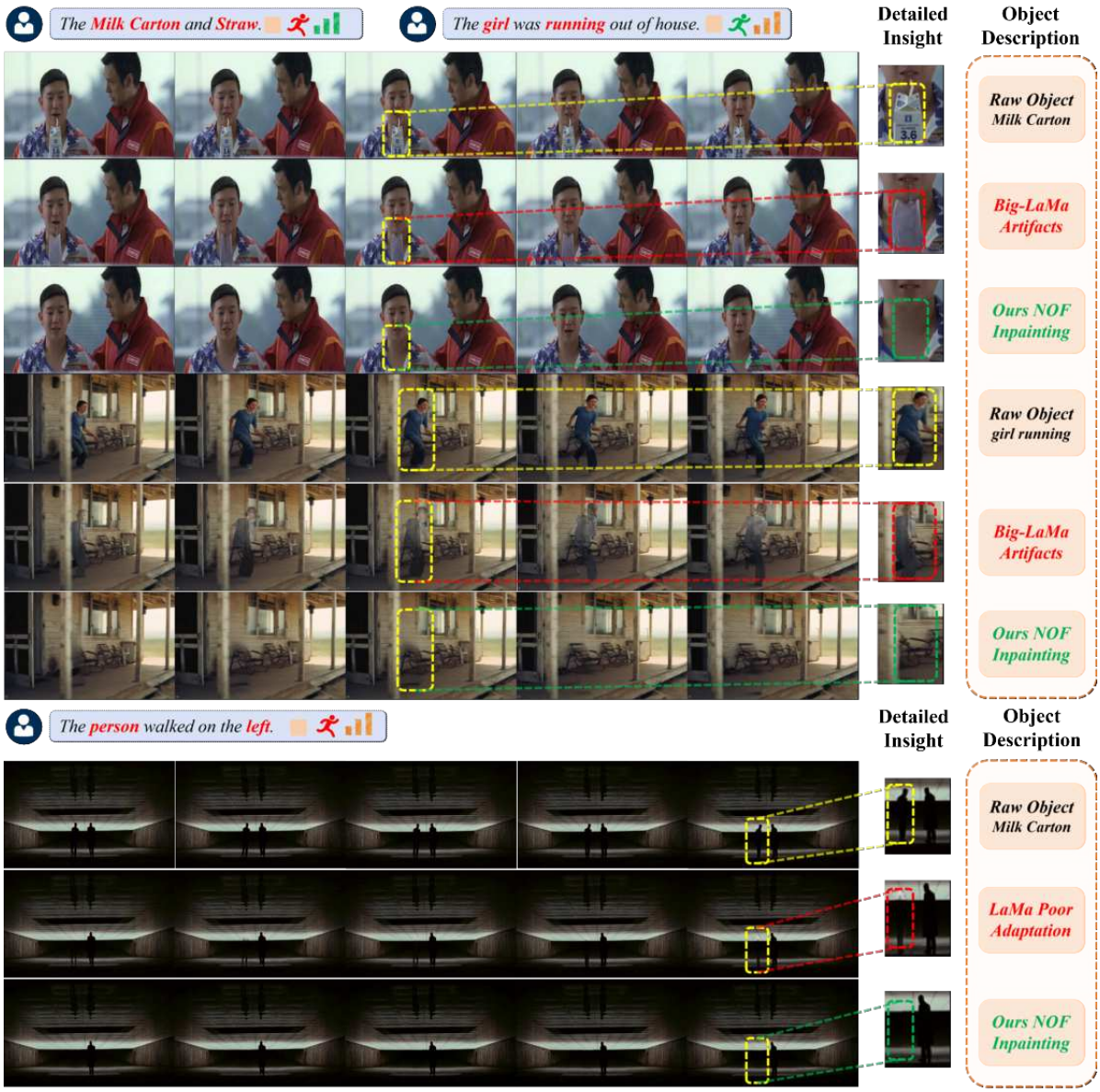


Figure 10. Visualization and details of fakeshade in experiment 0 produced by Big-LaMa

fast-moving targets, the model fails to maintain temporal consistency, leading to unnatural transitions and visual artifacts. And the visualization and details is as Figure. 10

The second issue is that Big-LaMa has poor adaptability. It performs poorly on noisy frames, low or rapidly changing lighting, and highly complex backgrounds. And the visualization is as Figure. And the visualization and details is as Figure. 10

Quantitative comparison shows that although Big-LaMa achieves good results in certain stable or simple scenes, its struggles to maintain temporal consistency across frames, and the generated results suffer from severe artifacts, especially in dynamic scenes or under challenging conditions, leading to a significant drop in visual quality (Δ Artifact Score = -146.5%).

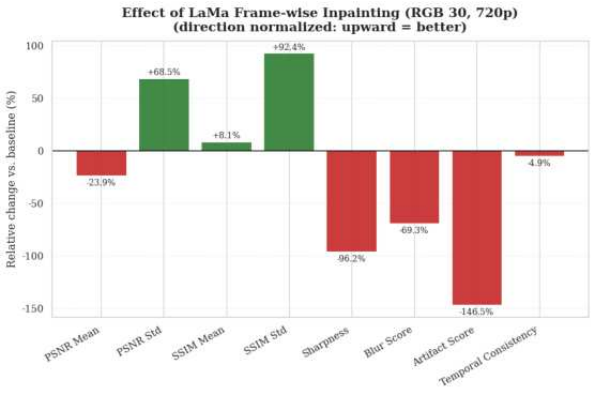


Figure 11. Visualization of quantitative metrics for Big-LaMa frame-wise inpainting

Table 1. LaMa4Video and PEANUT Comparision

Metrics	Value	PSNR/dB	PSNR(Std)/dB
Object(RGB30-720p)	**Value**	29.8239	2.4730
	Delta	**(-9.3684)**	**(-5.3802)**
Metrics	**Value**	**SSIM/dB**	**SSIM(Std)d/B**
Object(RGB30-720p)	**Value**	0.9560	0.0082
	Delta	**(+0.0719)**	**(-0.1001)**
Metrics	**Value**	**Sharpness**	**Blur**
Perceptual & P-Speed	**Value**	2.2208	48.8414
	Delta	**(-55.7792)**	**(+20.0002)**
Metrics	**Value**	**Artifact**	**TC**
Perceptual & P-Speed	**Value**	52.4213	78.8394
	Delta	**(+31.1528)**	**(-4.1007)**
Metrics	**Value**	**Runtime(s/f)**	
Perceptual & P-Speed	**Value**	11.5646	
	Delta	**(+11.2205)**	

4.2. P-MASk for Mask Generation

Our P-MASk Block is based on the backbone of Segment Anything Model 2 (SAM2) [9] and gain the idea from SAMWISE algorithm.Then, we add this algorithm and the proper SAM2 block to work for our P-MASk (Prompt-Guided Mask Generation).

SAM2 is a foundation model for promptable image and video segmentation, introduced by Meta AI in 2024. The model consists of four main modules: the Hiera image encoder, the Memory Attention Block (storing current frame information via cross-attention), the Mask Decoder & Prompt, and the Memory Encoder & Occlusion Head. Each frame is first encoded, then queried against historical memory, followed by mask generation based on prompts, and finally memory update, forming a temporal loop [10].

- **Unified image and video processing:** Treats images as single-frame videos and adopts a video-oriented architecture to handle both image and video tasks.
- **Streaming video segmentation:** Processes videos frame by frame, storing historical information in the memory block, enabling real-time interactive segmentation [9].
- **Enhanced interactive prompting:** Supports points, boxes, and masks. Achieves higher segmentation accuracy with fewer interactions in zero-shot RVOS/VOS tasks, outperforming SAM+XMem++ and SAM+Cutie.
- **Efficient encoder design:** Uses the Hiera-B hierarchical visual encoder, achieving slightly higher mIoU than SAM on image segmentation.
- **Large-scale video dataset support:** Accompanied by the Segment Anything Video (SA-V) dataset [11] with around 50k videos and over 30 million masks, significantly boosting model training capacity.

For our evaluation of video inpainting, following SAMWISE, we freeze SAM2 weights and use it as a black-box backbone. We wrap SAM2 with lightweight plugins, a Cross-Modal Temporal Adapter (CMT) and Conditional Memory Encoder (CME) [10], enabling text-driven feature-level temporal modeling. With minimal parameters, this transforms the original SAM2 + CMT + CME into a streaming RVOS model, without fine-tuning SAM2 weights or using an external large VLM.

Backbone	Visual Encoder	Text Encoder	Component		
SAM2.1	Hiera-B (89M)	CLIP-B (86M)	LLM	CMT (45M)	CME (45M)
✓	✓	✓	✓	×	✓

Table 2. Components and their status for SAM2.1 backbone.

Thus, SAMWISE consists of a frozen SAM2 backbone, a frozen text encoder, a Cross-Modal Temporal Adapter (CMT), a Conditional Memory Encoder (CME), and a small MLP for projecting text prompts.

The integration is based on a three-stream fusion. Notably, SAMWISE wraps around this pipeline without modifying the weights. The process consists of three streams: the visual stream, the text stream, and the tracking stream.

Visual Stream

In SAM2, the Hiera Image Encoder produces hierarchical features F for each frame independently. At each encoder layer, after generating frame embeddings, we insert the CMT visual branch. The process has two steps:

1.Stack the visual tokens across frames and feed them into CMT's Hierarchical Selective Attention (HSA) for cross-frame temporal modeling.

2.Apply cross-attention between the visual embeddings and text tokens (Text→Visual and Visual→Text) to obtain text-aware, temporally enhanced visual features.

The processed visual features are then fed back into the original SAM2 pipeline: Memory Attention → Memory Encoder → Mask Decoder.

Text Stream

The input sentence is tokenized and passed through the frozen text encoder to obtain text features E, and at each layer the CMT text branch performs cross-modal attention, where Visual-to-Text Attention (VTA) lets text tokens attend to visual features and Text-to-Visual Attention (TVA) lets visual tokens attend to text features, producing multimodal tokens in which both text and visual features are mutually aware.

From the adapted text features, we extract two key vectors: the [CLS] token as the Contextual Prompt E_C, representing overall semantics (e.g., "person in red clothes"), and the aggregated verb tokens as the Motion Prompt E_M, representing actions (e.g., "running" or "talking on the phone").

Project $[E_C, E_M]$ into the SAM2 prompt space using a learnable MLP. The resulting text prompt embedding is fed into SAM2's Mask Decoder.

Memory & Tracking Stream

We observed a tracking bias in SAM2: when the target is not clearly defined initially, SAM2 may select an incorrect object that loosely matches the text. To address this, we insert a Conditional Memory Encoder (CME) after SAM2's Memory Encoder, which receives the processed visual features.

To address the tracking bias in SAM2, where the model may select an object that does not match the text when the target is initially unclear, we introduce a lightweight Conditional Memory Encoder (CME). This module outputs a gating signal based on the match between the candidate object in the current frame and the text. It decides whether to update the Memory Bank and allows the model to refocus on the correct target.

We first modify the prompt side. SAM2 weights are frozen, and we use its utility functions to preprocess visual and text features. Text is encoded using a text encoder and an MLP to generate a prompt embedding. This embedding is fed directly to the Mask Decoder. This design is similar to a minimal RefSAM approach and integrates text guidance without changing the SAM2 core.

To improve cross-frame and cross-modal alignment, we add a lightweight Cross-Modal Temporal Adapter (CMT) before Memory Attention. It uses an extra Transformer block to perform cross-attention between multi-frame visual tokens and text tokens. The CMT acts as a pre-adapter outside SAM2 and does not modify its internal structure.

The CME module is activated when tracking bias is detected. It checks the match between the current frame mask and the text. If the existing memory object matches the text poorly and a better candidate exists, CME updates the corresponding memory slots. This ensures that the model tracks the correct object.

4.2.1. Quantitative Evaluation of P-MASk

This section will consist of four parts: explanation of hyperparameters, description of the quantitative evaluation metrics design, analysis of the first set of experimental tables, and analysis of the first set of visualizations.

Hyperparameters Selection

In this experiment, we selected two hyperparameters that directly affect the model's core performance: threshold and clip-window. These parameters influence the model's ability for cross-frame video understanding and mask generation.

The threshold converts the 0–1 probability masks produced by SAM2 and SAMWISE into binary segmentation masks, where each pixel's value represents foreground confidence. Since per-pixel computation scales quadratically with input size, images are cropped to 1024×1024, and the threshold balances recall and precision.

Threshold Settings	Effect on Mask Generation
Low-pass Threshold Filter (0.3–0.4)	Mask Area ↑, Light leakage ↑, Adhesion ↑
Optimal (0.5–0.55)	Official Evaluation: 0.5. Additional adapt 0.55.
High-pass Threshold Filter (0.6–0.7)	Mask Area ↓, Possible Edge Defects ↑, Small easily eliminated ↓

Table 3. Effects of different threshold settings on mask generation

Frame window size for evaluation (clip-window) indicates the length of the local temporal window fed into the model during inference, i.e., the number of frames processed at a time (as Table. 4).

Size	Effects on Mask Geometry	Effects on Spatial-Temporal Stability
Wide	Larger mask, light leakage, adhesion	Smoother ↑, Less jitter ↓, Robustness to motion ↑, Higher memory load ↑
Optimal	Official evaluation: default 8; for specific scenes:	Evaluation clip = 8 & CME decision = 4
Narrow	Smaller mask, possible edge defects	Occlusion or disappearance stability ↓, Sensitivity to rapid changes ↑, Instability ↑

Table 4. Effects of different clip window sizes on mask generation and spatial-temporal stability.

Model size indicates the number of parameters in the P-MASk model.

To verify the optimal configuration, we evaluated the model on a set of high-difficulty video frames, including scenarios with fast motion and rapid spatial transformations. Several statistics and evaluation metrics were selected for quantification. Since our task is related to video output, we summarize below the quantitative metrics used for different tasks.

Name / Metrics	Quality ↑	PSNR ↑	SSIM ↑	TC ↑	Temporal IoU ↑
P-MASk	✓	✓	✓	✓	✓
NOF-Eraser	×	✓	✓	✓	×

Name / Metrics	Sharpness ↑	Connectivity ↑	Blurry ↑	Artifact ↑	Time Cost ↑
P-MASk	✓	✓	×	×	×
NOF-Eraser	✓	×	✓	✓	✓

Table 5. Comparison of P-MASk and NOF-Eraser across all evaluation metrics, with all metric arrows colored red for clarity.

We selected frames from high-difficulty, fast-moving videos and provide a visualization of the masks produced by different model configurations.

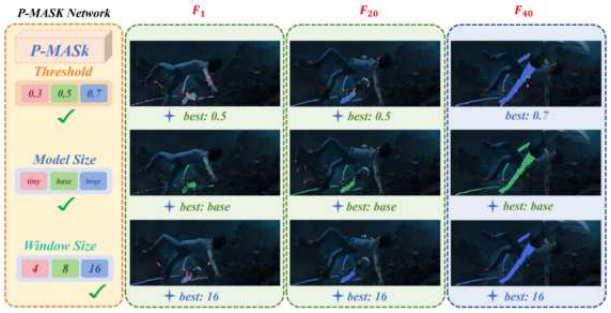


Figure 12. Visualzation for performance of different settings

Evaluation Metric Design

PSNR and SSIM account for 50% of the overall score and primarily measure the differences between the generated image and the original in terms of pixel values and structure. PSNR, measured in decibels (dB), increases as the image becomes more similar to the original. SSIM evaluates not only pixel values but also luminance, contrast, and structural information, ranging from 0 to 1, with higher values indicating stronger structural similarity.

The formula for Peak Signal-to-Noise Ratio (PSNR) is:

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) \tag{6}$$

where:

- MAX is the maximum possible pixel value of the image .
- MeanSquaredError (MSE) is the average of the squares of the differences in pixel values between the original and the reconstructed image.

In practice, PSNR and SSIM exhibit a trade-off: optimizing PSNR reduces pixel-level errors and ensures overall object correctness and contour accuracy, while some local structural details may be lost. We believe that this strategy of prioritizing overall correctness before refining details is effective for mask generation.

Next, temporal consistency (TC) accounts for 35% of the overall score and is the most important metric for frame-by-frame mask generation. TC measures the stability of the generated masks, focusing on smoothness, absence of flickering or jitter, and consistent object coverage, ensuring coherent mask propagation throughout the video.

Finally, the Sobel sharpness metric (Sobel_sharpness) accounts for 15% of the overall score and evaluates edge clarity. It computes a scalar by aggregating per-pixel values of the gradient magnitude map M. Higher values indicate clearer images and sharper edges while avoiding excessive spurious edges introduced by the model. For efficiency and speed, the metric is implemented using the mean squared gradient magnitude (MSGM).

The formula for image sharpness is defined as follows:

$$\text{Sharpness}_{\text{Mean}} = \frac{1}{W \cdot H} \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} \mathbf{M}(x, y) \tag{7}$$

Analysis of the First Experimental Table

Table 6. Image Quality and Geometric Structure Evaluation. Combined Score represents PSNR↑ : SSIM↑ : Sobel_Sharpness↑ : TC↑ with weights 6:4:3:7.

Combined Score (ID-Config)	Geometric Structure Metrics		
	Temporal-IoU ↑	Edge Sharp ↑	Connect ↑
70.83 (I)	0.7623 **(-0.0206)**	0.5383 **(-0.0412)**	0.5213 **(+0.0282)**
75.49 (II)	0.7829	0.5795	0.4931
67.08 (III)	0.7046	0.4957	0.6753
69.05 (IV)	0.7257	0.5264	0.5671
67.83 (V)	0.7595	0.5281	0.6167
60.94 (VI)	0.7492	0.0470	0.6284
68.48 (VII)	0.7543	0.0540	0.6714

From the data, although the default base setting achieves a score of 70.8393, which is lower than the best model-large with Text Encoder RoBERTa (75.4921), we prioritize a trade-off between computational efficiency and output quality. Since the core task focuses on video object erasure in the NOF-Eraser module, where mask quality is

Table 7. Model Configuration for Each Test

ID	Model-Size	ClipWindow	Threshold
I	**Base**	**8 (2x2x2)**	**base (0.5)**
II	Large	8 (2x2x2)	base (0.5)
III	Tiny	8 (2x2x2)	base (0.5)
IV	Base	16 (4x2x2)	base (0.5)
V	Base	4 (1x2x2)	base (0.5)
VI	Base	8 (2x2x2)	loose (0.3)
VII	Base	8 (2x2x2)	strict (0.7)

secondary, the base model is selected to balance efficiency and quality.

Temporal IoU evaluates the model's consistency and accuracy over time. Compared to standard IoU, it emphasizes temporal correctness, measuring whether the generated masks accurately follow object motion across frames. A higher Temporal IoU indicates precise mask tracking and temporal stability. The base model achieves a Temporal IoU of 0.7623, ensuring good frame-to-frame consistency.

Edge clarity is measured using the Root Mean Square Gradient Magnitude (RMS), which maintains sharp edges while avoiding spurious ones. The base model achieves 0.5383, preserving clear edges without introducing artifacts.

Connectivity shows an interesting trend. The tiny model achieves the highest connectivity (0.6753) due to its focus on low-level features such as edges, textures, and shapes, which are critical for spatial consistency. However, its other quality metrics are poor. The base model balances quality, connectivity, and computational cost, achieving 0.5375. Large models focus on high-level features and fine details, which can introduce artifacts and disrupt temporal consistency, reducing connectivity.

Analysis of the First Visualization

From (a), (b), (d), and (f) in Figure.13, we observe that different model hyperparameters, including Model Size, Clip Window and Threshold, affect mask generation in terms of geometric structure and spatio-temporal continuity. From (c), we further see that the mask generation threshold has a stronger impact on temporal consistency (Temporal IoU & Connectivity, sensitivity > 0.7) than on target geometry (sensitivity < 0.54).

Based on the above analysis, we verify that the base settings proposed by the authors achieve the most effective balance among mask quality, computational cost, and temporal consistency, with the evaluated best settings being Model Size: base, Clip Window: 8, and Threshold: 0.5.

4.2.2. NOF-Eraser for Video Inpainting

This section will also consist of four parts: explanation of hyperparameters, description of the quantitative evaluation metrics design, analysis of the second set of experimental ta- bles, and analysis of the second set of visualizations.

Hyperparameters Selection

In this experiment, we selected two hyperparameters that significantly affect the performance of the NOF-Eraser model: neighbor stride and step. Both directly influence the model's temporal information extraction, the modeling of relationships between adjacent frames, and the video frame generation process.

The neighbor stride hyperparameter determines the interval for selecting adjacent frames, controlling how many neighboring frames the model uses as references to assist in reconstructing the current frame.

Neighbour-Stride Settings	Effect to Video Inpainting
Low-Stride (2–4)	Further Frames ↓, TC (Timing Consistency) ↑, Local Details ↑, Large-scale movement ↓
Optimal (5–8)	Official Evaluation: Keep default as 5. Additionally adapt 6–8 for specific scenes
High-Stride (10+)	Further Frames ↑, TC (Timing Consistency) ↓, Local Details ↓, Large-scale movement ↑

Table 8. Effect of Neighbour-Stride Settings on Video Inpainting

In video inpainting tasks, performing full inpainting on every frame is computationally expensive. Consecutive frames exhibit high temporal redundancy, so we adopt a step-based sparse sampling strategy.

Given a step size n and a video sequence $F = \{f_1, f_2, \ldots, f_T\}$, the output is

$$\text{Output}(F) = \begin{cases} \text{Model}(f_{1+Kn}), & \text{for key frames } f_{1+Kn}, \\ \text{Propagate}(f_i), & \text{for non-key frames } f_i, \end{cases} \tag{8}$$

where non-key frames are generated via a lightweight flow-guided propagation module using previously inpainted frames, avoiding full inpainting computation.

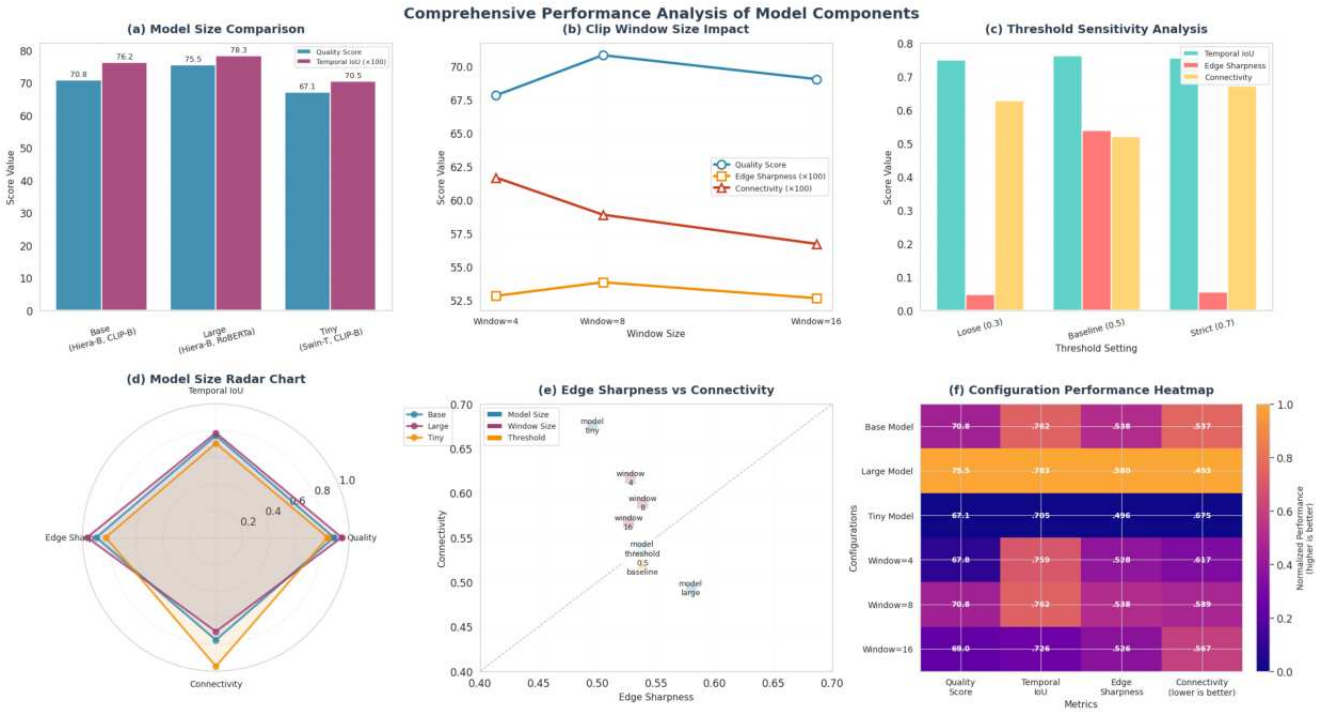


Figure 13. Impact of Model Hyperparameters on Mask Geometry and Temporal Consistency

Step Settings	Effect to Video Inpainting
Dense Inpaint (n=1)	Restore Details ↑↑, Slow Change ↑↑, Rapid Motion ↓↓, TC ↑↑, Temporal Redundancy ↑ ↑, Computation ↑ ↑
Low-Step (n=2–4)	Restore Details ↑, Slow Change ↑, Rapid Motion ↓, TC ↑, Temporal Redundancy ↑, Computation ↑
Optimal (n=5–10)	Official Evaluation: Keep default as 10 as official settings. Additionally adapt 5–9 for specific scenes
High-Step (n=10+)	Restore Details ↓, Slow Change ↓, Rapid Motion ↑, TC ↓, Temporal Redundancy ↓, Computation ↓

Table 9. Effect of Step Settings on Video Inpainting

Model Settings			Best Balance	Video Settings	
Configuration	Neighbor Stride	Step	Choice	Modality & Fps	Reference Number
neighbor_3	3	10	×	RGB 30 (720p)	-1 (default all)
neighbor_5	5	10	✓	RGB 30 (720p)	-1 (default all)
neighbor_10	10	10	×	RGB 30 (720p)	-1 (default all)
step_5	5	5	×	RGB 30 (720p)	-1 (default all)
step_10	5	10	✓	RGB 30 (720p)	-1 (default all)
step_20	5	20	×	RGB 30 (720p)	-1 (default all)

Table 10. Base Parameter Settings Table

Evaluation Metric Design

PSNR is 39.1923 when using the base configuration (neighbor stride = 5, step = 10). Although this value is lower than that obtained with the high configuration (neighbor stride = 10, step = 10), the model demonstrates higher stability, with $\Delta\text{PSNR(std)} = -2.8886$, indicating that the base configuration is more robust across different video scenarios.

The Blur Score and Artifact Score are based on frequency-domain analysis and evaluate image sharpness and artifacts. The process involves converting the restored image to whitescale, performing a Fourier transform to obtain the frequency domain, computing the spectrum magnitude, and filtering high-frequency pixels using a threshold. The sum of the selected high-frequency regions provides a sharpness score, which is then converted to the final metrics using the formulas

$$\text{Blur Score} = 100 - \text{sharpness score} \tag{9}$$

$$\text{Artifact Score} = 100 - \phi \times \text{sharpness score} \tag{10}$$

This approach leverages the correlation between high-frequency components and visual quality, enabling quantitative evaluation of restored images.

Analysis of the Second Experimental Table

Table 11. Experiment2-Base Parameter Comparision Evaluation

Image Fidelity	Model Config	PSNR (Mean) / dB ↑	PSNR (Std) / dB ↑
PSNR Stability	(n, s) = (3,10)	38.8319 (-0.3604)	9.4814
	(n, s) = (5,10)	39.1923 (base)	7.8532
	(n, s) = (10,10)	40.8194 (+1.6271)	10.7418
	(n, s) = (5,5)	37.7419 (-1.4504)	8.7414
	(n, s) = (5,10)	39.1923 (base)	7.8532
	(n, s) = (3,20)	38.0184 (-1.1739)	9.8414
Image Fidelity	**Value**	**SSIM (Mean) / dB↑**	**SSIM (Std) / dB ↑**
SSIM Stability	(n, s) = (3,10)	0.8718 (-0.0123)	0.1083
	(n, s) = (5,10)	0.8841 (base)	0.0984
	(n, s) = (10,10)	0.8765 (+0.0076)	0.0921
	(n, s) = (5,5)	0.8673 (-0.0168)	0.0905
	(n, s) = (5,10)	0.8841 (base)	0.0984
	(n, s) = (3,20)	0.8692 (-0.0149)	0.1056
Evaluation Indicators	**Value**	**Sharpness Score ↑**	**Blur Score ↑**
Perceived Quality Geometric Indicators	(n, s) = (3,10)	58 (-6)	30.4814 (+1.6402)
	(n, s) = (5,10)	64 (base)	28.8412 (base)
	(n, s) = (10,10)	69 (+5)	29.4673 (+0.6261)
	(n, s) = (5,5)	58 (-6)	29.9841 (+1.1429)
	(n, s) = (5,10)	64 (base)	28.8412 (base)
	(n, s) = (3,20)	63 (-1)	28.8955 (+0.0543)
Evaluation indicators	**Value**	**Artifact Score ↑**	**Temporal Consistency ↑**
Perceived Quality & Temporal-Geometric Metrics	(n, s) = (3,10)	22.4213 (+1.1528)	78.8394 (-4.1007)
	(n, s) = (5,10)	21.2685 (base)	82.9401 (base)
	(n, s) = (10,10)	22.8913 (+1.6228)	84.4885 (+1.5484)
	(n, s) = (5,5)	21.4254 (-0.156)	80.0348 (-2.9053)
	(n, s) = (5,10)	21.2685 (base)	82.9401 (base)
	(n, s) = (3,20)	21.2676 (-0.0009)	82.4819 (-0.4582)
Runtime (s/vsec)	**Model Config**	**30fps ↑**	**60fps ↑**
One Second Video to the amount of time cost processing (second)	(n, s) = (3,10)	5.526 (-4.797)	11.052(-9.594)
	(n, s) = (5,10)	10.323 (base)	20.646 (base)
	(n, s) = (10,10)	20.493 (+10.17)	40.986 (+20.34)
	(n, s) = (5,5)	8.523 (-1.8)	17.046 (-3.6)
	(n, s) = (5,10)	10.323 (base)	20.646 (base)
	(n, s) = (3,20)	17.586 (+7.263)	35.172 (+14.526)

In terms of video fidelity and restoration stability metrics, *i.e.*, PSNR (mean | std) and SSIM (mean | std), the base configuration (neighbor stride = 5, step = 10) achieves both high restoration quality and stability. Temporal consistency is also favorable, with a TC score of 82.9401.

We further compared the model's performance across different video inputs. We observed that using larger values for both neighbor stride and step yields the best restoration quality and temporal consistency, surpassing the recommended base configuration (ΔPSNR = +1.6271, ΔTC = +1.5484). However, this improvement comes at the cost of increased computational time.

For videos with varying frame rates, the per-second processing time and the corresponding real-world computational overhead are summarized as follows:

Analysis of the Second Visualization

From the Figure. 14, we observe that (a) shows the time cost under different model configurations, while (b) illustrates the increase in time cost when processing videos at different frame rates (fps30 and fps60). Comparing (a) and (d), we see that increasing the model inference step has a smaller impact on time cost than increasing the neighbor stride. From (c), by balancing the key metrics—including restored image quality (PSNR), temporal consistency (TC), and computational time—we determine that the best practical configuration is a neighbor stride of 5 and a step of 10.

5. Conclusion and Future Work

In this paper, we present a fully automated end-to-end framework for video object moval that addresses key limitations of existing approaches in semantic understanding, spatio-temporal consistency, and practical deployability. By unifying text-driven guidance with temporally coherent video restoration, our method provides a controllable and scalable solution for real-world video editing tasks.

At the core of our framework, we integrate the proposed P-MASK module with a Conditional Memory Encoder (CME) to automatically generate accurate and temporally consistent object masks from user-provided textual instructions, effectively removing the need for manual annotations. This design enables robust object localization across video sequences while maintaining semantic alignment with high-level user intent.

To ensure visually realistic and temporally stable background reconstruction, we introduce NOF-Eraser, a flow-guided end-to-end restoration module that jointly models motion completion, feature propagation, and content synthesis. By explicitly leveraging temporal dependencies, NOF-Eraser mitigates common artifacts such as flickering and inconsistency, even in challenging motion scenarios.

Furthermore, our framework is designed with a strong emphasis on efficiency and quality balance, enabling high-quality video object removal with practical computational requirements. Extensive experiments demonstrate that the proposed approach consistently outperforms conventional video inpainting methods, particularly in scenarios involving complex semantics and long-range temporal dependencies.

Future work will focus on introducing three-dimensional geometric consistency to improve visual realism in complex motion scenarios, and on optimizing model structures and inference processes to support real-time processing of high-resolution videos. In addition, future studies will explore causal reasoning and dynamic prediction, as well as extensions to more advanced video editing tasks such as text-driven editing, multi-object manipulation, and cross-modal interaction.

References

[1] C. Wu, X. Li, and Z. Zhang, "Language as queries for referring video object segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12 345–12 354.

[2] W. Quan, J. Chen, Y. Liu, D.-M. Yan, and P. Wonka, "Deep learning-based image and video inpainting: A survey," *International Journal of Computer Vision*, 2024, arXiv preprint arXiv:2401.03395. [Online]. Available: https://arxiv.org/abs/2401.03395

[3] Z. Li, C.-Z. Lu, J. Qin, C.-L. Guo, and M.-M. Cheng, "Towards an end-to-end framework for flow-

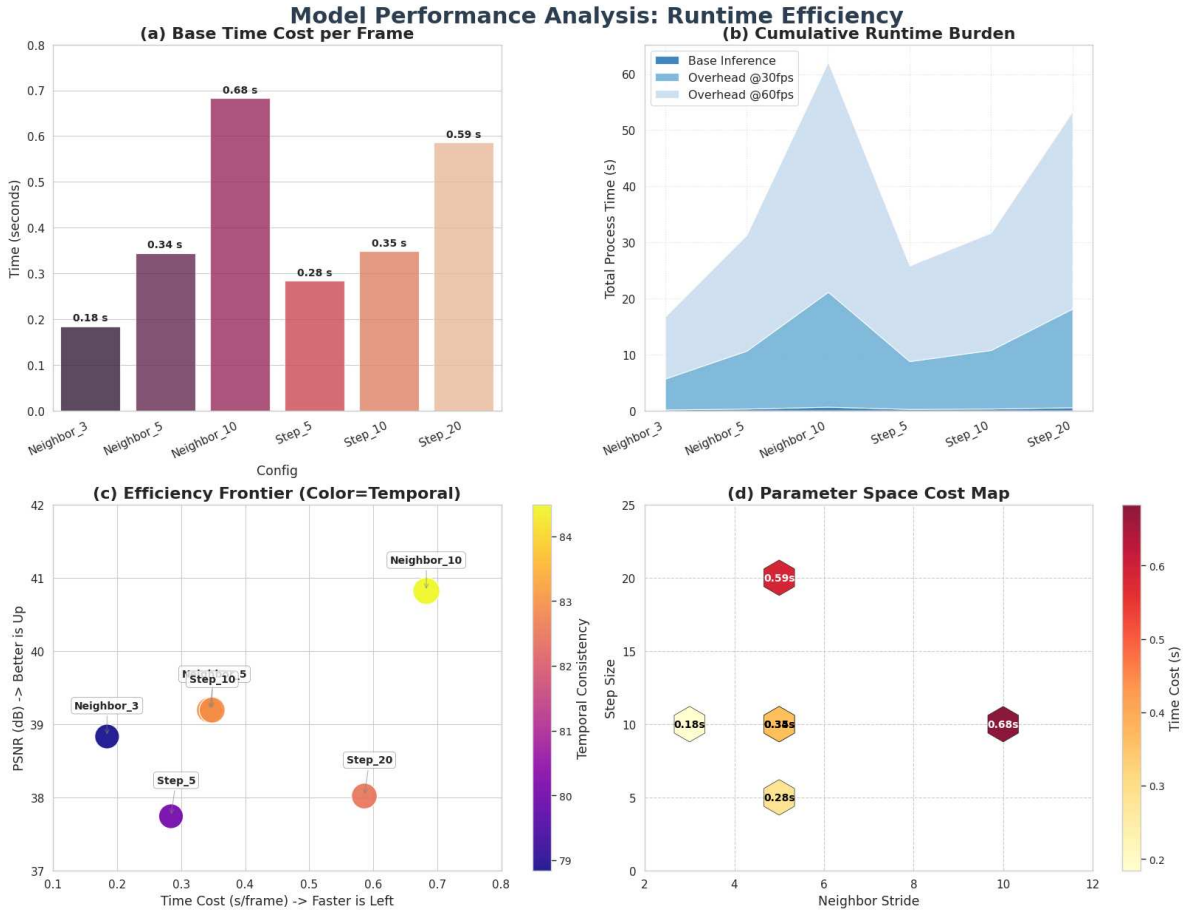


Figure 14. Model Performance Analysis of Runtime Efficiency

guided video inpainting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17 562–17 571. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/papers/Li_Towards_an_End-to-End_Framework_for_Flow-Guided_Video_Inpainting_CVPR_2022_paper.pdf

[4] X. Wang, K. C. K. Chan, K. Yu, C. Dong, and C. C. Loy, "Edvr: Video restoration with enhanced deformable convolutional networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019, pp. 1954–1963, video restoration including super-resolution and deblurring; wins NTIRE19 challenges.

[5] M. Gao, "Deep learning for video object segmentation: a review," *Artificial Intelligence Review*, 2022, discusses limitations of common VOS datasets such as DAVIS and YouTube-VOS. [Online]. Available: https://link.springer.com/article/10.1007/s10462-022-10176-7

[6] J. Xu and L. Yang, "Youtube-vos: A large-scale video object segmentation benchmark," in *arXiv preprint arXiv:1809.03327*, 2018. [Online]. Available: https://arxiv.org/pdf/1809.03327

[7] A. Ranjan and M. J. Black, "Optical flow estimation using a spatial pyramid network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4161–4170.

[8] Z. Zhang, Z. Shi, J. Wu, and D. Lin, "Basicvsr++: Enhancing video super-resolution via spatio-temporal convolution," *arXiv preprint arXiv:2204.02663*, 2022.

[9] Meta AI, "SAM 2: A new approach to video segmentation," https://ai.meta.com/sam2/, 2023.

[10] C. Cuttano, G. Trivigno, G. Rosi, C. Masone, and G. Averta, "Samwise: Infusing wisdom in sam2 for text-driven video segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 3395–3405.

[11] Meta AI, "Segment anything video (SA-V) dataset," https://ai.meta.com/datasets/segment-anything-video/, 2024.